

Kapitel 4

Ausdrücke und Operatoren

4.1 Aufbau von Ausdrücken

Ausdrücke setzen sich aus *Operanden*, *Operatoren* und *runden Klammern* zusammen, wobei die Operanden in der Regel Konstanten, Variablen (einschließlich Feldkomponenten) und Funktionsaufrufe sein können. Die einfachsten Ausdrücke bestehen nur aus einem Operanden.

An verschiedenen Stellen sind wir bereits Ausdrücken begegnet, ohne dass ihnen besondere Aufmerksamkeit geschenkt wurde:

<code>i</code>	als Argument von <code>printf</code>
<code>i - 1</code>	bei der Schleifenzählung
<code>v1[i] * v2[i]</code>	beim Skalarprodukt
<code>i < LAENGE</code>	beim Skalarprodukt

Einige weitere Ausdrücke als Beispiele:

```
(a + b) * 7
(a + b) * (c + d)
- a
```

Wie schon erwähnt, sind die Variablen, die wir in den Beispielen als Argumente für `printf` verwendet haben, formal als spezielle Ausdrücke anzusehen. Entsprechend können wir natürlich auch beliebige Ausdrücke als Argumente für `printf` verwenden. Warum das bei `scanf` nicht möglich ist, werden wir uns noch ansehen.

4.2 Die Wertzuweisung

Eine Besonderheit von C gegenüber anderen Sprachen ist, dass auch der *Zuweisungsoperator* (`=`) als Operator betrachtet wird:

```
i = i - 1
```

ist also formal ein Ausdruck und wird, wie in unseren Beispielen geschehen, nur durch das nachgestellte Semikolon zu einer Anweisung. Die Wirkung: Zunächst wird `i - 1` berechnet, dann der resultierende Wert der Variablen `i` zugewiesen. Dieser Wert ist gleichzeitig der Wert des gesamten Ausdrucks.

Damit werden Ausdrücke wie

```
i = (j = (k = 0))
```

möglich: Die drei Variablen `i`, `j` und `k` erhalten jeweils den Wert 0. Da mehrere Gleichheitszeichen von rechts nach links abgearbeitet werden, sind die Klammern in diesem Ausdruck sogar überflüssig und wir können kurz schreiben

```
i = j = k = 0
```

Compiler betrachten den Namen einer Variablen in der Regel als Repräsentanten für den Wert der Variablen. Bei der Funktion `scanf` waren wir deshalb gezwungen, den Argumenten den Adressoperator `&` voranzustellen, damit die Adresse des Arguments anstelle seines Wertes übergeben wird. Gleiches gilt eigentlich auch für die Wertzuweisung: Benötigt wird ja nicht der Wert der Variablen, die verändert werden soll, sondern ihre Adresse.

Da es klar ist, dass auf der linken Seite des Zuweisungsoperators eine Adresse benötigt wird, und da die explizite Angabe des Adressoperators an allen diesen Stellen viel zu mühsam wäre, vereinbart man ganz einfach: Auf der linken Seite des Zuweisungsoperators repräsentiert ein Variablenname die Adresse der Variablen und nicht wie sonst ihren Wert. Schreiben wir etwa den (in der Sache unsinnigen) Ausdruck

```
i = i
```

so steht also `i` links und rechts vom Zuweisungsoperator für ganz verschiedene Dinge.

In der Regel gibt das keine Probleme. Wesentlich wird die Unterscheidung allerdings, wenn man sich überlegt, was aus der linken Seite eines Zuweisungsoperators stehen darf und was nicht. Dass Variablen und Elemente von Feldern erlaubt sind, haben wir bereits gesehen. Nicht erlaubt sind dagegen zum Beispiel

```
5 = i          /* nicht */
i + j = k      /* erlaubt */
```

da weder die Konstante 5 noch der Ausdruck `i + j` einen eigenen Speicherplatz mit Adresse besitzen.

Offensichtlich unterliegen die Argumente von `scanf` denselben Regeln wie die linken Operanden von Wertzuweisungen: `scanf` soll ja auch den Variablen Werte zuweisen.

4.3 Arithmetische Operatoren

C kennt fünf *arithmetische Operatoren*, nämlich `+`, `-`, `*`, `/` und `%`.

Plus und *Minus* gibt es in zwei Varianten: Als *unäre* Operatoren dienen sie als Vorzeichen, wobei das positive Vorzeichen erst durch den Standard vorgeschrieben wurde; als *binäre* Operatoren bewirken sie Addition bzw. Subtraktion.

Multiplikation bewirkt der Stern, *Division* der Schrägstrich. Bei der Division ganzzahliger Operanden ist eine Besonderheit zu beachten: Der Quotient ist wieder ganzzahlig! Als Ergänzung zum Divisionsoperator gibt es deshalb den *Modulo-Operator* `%`, der den Rest bei der Division ganzer Zahlen liefert. Dieser Operator darf *nur* für ganzzahlige Operanden verwendet werden, nicht für Gleitkommawerte. Wir sehen uns einige Beispiele für die Division an:

```
10.0 / 3  ergibt  3.333333
 10 / 3  ergibt   3
 10 % 3  ergibt   1
```

Der *Divisionsrest* ist nur dann eindeutig bestimmt, wenn beide Operanden nichtnegativ sind; der Divisionsrest ist dann positiv. Bei mindestens einem negativen Operanden ist es den speziellen Implementationen freigestellt, ob der Divisionsrest positiv oder negativ ist. Entsprechend ist auch die ganzzahlige Division nur für nichtnegative Operanden eindeutig, denn die Beziehung

```
a == (a / b) * b + a % b
```

muss für beliebige ganzzahlige Werte *a* und *b* (*b* ungleich 0!) erfüllt sein. Möglich ist zum Beispiel

```
10 / - 3 == - 3
```

```
10 % - 3 == 1
```

ebenso wie

```
10 / - 3 == - 4
```

```
10 % - 3 == - 2
```

Arithmetische Operatoren werden von links nach rechts abgearbeitet. Dabei werden die Regel „Punktrechnung vor Strichrechnung“ natürlich ebenso berücksichtigt wie Klammern. Außerdem besitzen Plus und Minus als unäre Vorzeichenoperatoren höhere Priorität als die binären Operatoren. Man hat also zum Beispiel

```
a + b * c == a + (b * c)
a + b - c == (a + b) - c
- a * b == (- a) * b
```

Auf die Probleme numerischen Rechnens wird in einem eigenen Kapitel noch eingegangen.

4.4 Inkrementierung und Dekrementierung

C verfügt über spezielle Operatoren zum *Inkrementieren* und *Dekrementieren* von Variablen: `++` erhöht seinen Operanden um 1, `--` verringert ihn um 1.

Beide Operatoren können wahlweise in *Präfix*-Schreibweise (vor dem Operanden) oder auch *Postfix*-Schreibweise (hinter dem Operanden) verwendet werden. Äquivalent sind also jeweils die Anweisungen

```
++i;          i++;          i = i + 1;
--i;          i--;          i = i - 1;
```

Als Ausdrücke sind Präfix- und Postfix-Schreibweise keineswegs äquivalent:

- Bei Präfix-Schreibweise wird *erst* inkrementiert bzw. dekrementiert und *danach* der bereits veränderte Wert der Variablen als Wert des Ausdrucks genommen.
- Bei Postfix-Schreibweise wird *erst* der noch unveränderte Wert der Variablen als Wert des Ausdrucks genommen und *danach* inkrementiert bzw. dekrementiert.

So hat nach

```
i = 10; j = ++i;
```

i den Wert 11 und *j* ebenfalls den Wert 11. Nach

```
i = 10; j = i++;
```

hat *i* wiederum den Wert 11, *j* jedoch den Wert 10.

Interessant sind beide Operatoren vor allem im Zusammenhang mit Schleifen und Vektoren. Das Beispiel, in dem wir zwei Vektoren gelesen und ihr Skalarprodukt berechnet haben, können wir mit ihnen durchaus kompakter schreiben:

```
#include <stdio.h>

#define LAENGE 5

int main(void) {
```

```

double v1[LAENGE], v2[LAENGE], prod;    /* Variablendekl. */
int i;

printf("1. Vektor eingeben!\n");        /* 1. Vektor lesen */
i = 0;                                  /* Schleife Variante 1 */
while (i < LAENGE) {
    scanf("%lf", &v1[i]);
    i++;
}

printf("2. Vektor eingeben!\n");        /* 2. Vektor lesen */
i = - 1;                                /* Schleife Variante 2 */
while (++i < LAENGE) {
    scanf("%lf", &v2[i]);
}

/* Skalarprodukt berechnen und ausgeben */
i = 0;                                  /* Schleife Variante 3 */
prod = 0;
while (i++ < LAENGE) {
    prod = prod + v1[i-1] * v2[i-1];
}
printf("Das Skalarprodukt ist: %f\n", prod);

return 0;
}

```

Quelltext 4.1: Inkrement-Operator

Es sollte klar sein, dass für die Operanden von Inkrementierung und Dekrementierung dieselben Regeln gelten wie für die Operanden auf der linken Seite eines Zuweisungsoperators: Erneut wird ja eine Adresse benötigt, an der der veränderte Wert gespeichert werden soll.

In der Regel wird man Inkrementierung und Dekrementierung für ganzzahlige Operanden verwenden; erlaubt sind sie aber auch für Gleitkommaoperanden.

4.5 Vergleichsoperatoren

C kennt die sechs *Vergleichsoperatoren* `==`, `!=`, `<`, `<=`, `>` und `>=`. Das Resultat eines solchen Ausdrucks ist *wahr* (gleich 1), wenn die Operanden in der Relation zueinander stehen, die der Operator beschreibt, und *falsch* (gleich 0) sonst. Letztlich sind die Resultate von Vergleichen also `int`-Werte; einen besonderen Typ *boolean* und entsprechende logische Konstanten kennt C nicht. Es steht jedem Programmierer natürlich frei, mit `enum` und `typedef` die logischen Konstanten `false` und `true` sowie einen „Typ“ *boolean* geeignet zu definieren.

Da Vergleichsausdrücke `int`-Werte sind, können sie Operanden in arithmetischen Ausdrücken sein. Da die Vergleichsoperatoren eine geringere *Priorität* als die arithmetischen Operatoren haben, kommt man in der Regel ohne Klammern aus. Der Ausdruck

```
a < b + c
```

entspricht also

```
a < (b + c)
```

a	! a	a	b	a && b	a	b	a b
0	1	0	0	0	0	0	0
≠ 0	0	≠ 0	0	0	≠ 0	0	1
		0	≠ 0	0	0	≠ 0	1
		≠ 0	≠ 0	1	≠ 0	≠ 0	1

Tabelle 4.1: Wahrheitstafeln für logische Operatoren

was man in der Regel auch beabsichtigen wird. Die Prioritäten werden im einzelnen noch einmal behandelt, wenn wir mehr Operatoren kennen.

Auf etwas anderes soll an dieser Stelle aber noch kurz hingewiesen werden: Wie bereits erwähnt, kann es verhängnisvoll sein, wenn man statt `==` nur `=` schreibt. Jetzt können wir klären, warum das so ist. Dazu betrachten wir das folgende Beispiel:

```
if (i = 1) { /* immer wahr */
    ...
}
else {
    ...
}
```

Hier wird die Anweisungsfolge, die dem `else` folgt, *nie* ausgeführt! Bei der Auswertung von `i = 1` erhält die Variable `i` den Wert 1. Auch der Wert des Ausdrucks insgesamt ist damit 1, wird entsprechend als *wahr* interpretiert und bewirkt die Ausführung der unmittelbar folgenden Anweisungen.

Auch Endlosschleifen kann man so bequem (und ungewollt) „zusammenbasteln“:

```
while (i = 1) { /* Endlosschleife */
    ...
}
```

Änderungen von `i` im Schleifenrumpf haben hierauf keinen Einfluss, da der Ausdruck `i = 1` vor jedem Schleifendurchgang erneut ausgewertet wird.

4.6 Logische Operatoren

Auch wenn C, wie wir gesehen haben, keinen Typ `boolean` kennt, kennt es doch drei *logische Operatoren*: `&&` (*und*), `||` (*oder*) und `!` (*nicht*).

Anwenden darf man diese Operatoren auf beliebige Operanden, die sich mit 0 vergleichen lassen – also insbesondere Zahlwerte, da wieder nach der bereits bekannten Regel vorgegangen wird: Ein Wert wird als *wahr* interpretiert, wenn er ungleich Null ist, und als *falsch*, wenn er gleich Null ist. Das Resultat ist entweder 1 oder 0. Die *Wahrheitstafeln* der drei logischen Operatoren finden Sie in Tabelle 4.1.

Logische Ausdrücke werden von links nach rechts abgearbeitet, wobei die Prioritäten der Operatoren berücksichtigt werden (erst die Negation, dann *und*, zuletzt *oder*). Eine Besonderheit gegenüber anderen Programmiersprachen legt der Standard ausdrücklich fest: Die Auswertung eines logischen Ausdrucks wird beendet, sobald das Ergebnis feststeht. Das ist gelegentlich ausgesprochen nützlich. Im Ausdruck

```
(d != 0) && (c / d < 10)
```

wird so die Division *nicht* ausgeführt, wenn `d` gleich 0 ist: Der erste Vergleich stellt fest, dass das Resultat *falsch* ist, unabhängig vom Wert des anderen Operanden – und verhindert so die in diesem Falle nicht zulässige Division. Man spricht dabei von „*lazy evaluation*“.

4.7 Zusammengesetzte Zuweisungen

Neben dem einfachen Zuweisungsoperator `=` gibt es in C zehn *zusammengesetzte Zuweisungsoperatoren*. Diese haben einheitlich die Form `op=`, wobei `op` ein binärer Operator ist. Zwischen dem binären Operator und dem Gleichheitszeichen *darf kein* Leerzeichen stehen (eine Neuerung des Standards). Für eine Variable `var` und einen Ausdruck `ausdr` kann man die Wirkung der zusammengesetzten Zuweisungsoperatoren so beschreiben:

`var op= ausdr`

entspricht

`var = var op ausdr`

d.h. zunächst wird der Ausdruck `ausdr` „normal“ ausgewertet, dann sein Resultat mit dem Wert der Variablen `var` durch den Operator `op` verknüpft und schließlich dieses Resultat in der Variablen `var` gespeichert. Der gespeicherte Wert ist gleichzeitig auch der Wert des gesamten Ausdrucks.

Nicht alle binären Operatoren sind bei der Zusammensetzung erlaubt; von den Operatoren, die wir bislang kennengelernt haben, dürfen nur die fünf arithmetischen Operatoren verwendet werden. Sehen wir uns einige konkrete Beispiele an. Die nebeneinanderstehenden Ausdrücke sind hier jeweils äquivalent:

<code>x += 3.5</code>	entspricht	<code>x = x + 3.5</code>
<code>i *= j + 10</code>	entspricht	<code>i = i * (j + 10)</code>
<code>a += b += c</code>	entspricht	<code>a = a + (b = b + c)</code>
<code>v[i + j*k] += 3</code>	entspricht	<code>v[i + j*k] = v[i + j*k] + 3</code>

Insbesondere das letzte dieser Beispiele zeigt, dass die zusammengesetzten Zuweisungsoperatoren einige Schreibarbeit sparen helfen. Aber nicht nur das: Dem Compiler wird es erleichtert, effizienten Code zu erzeugen. Der Indexausdruck im letzten Beispiel braucht auf jeden Fall nur einmal berechnet zu werden. Verwendet man die zusammengesetzte Zuweisung, so steht der Indexausdruck auch nur einmal im Programm, so dass der Compiler ihn nicht doppelt berechnen muss. Verwendet man dagegen die einfache Zuweisung, so muss man darauf hoffen, dass der Compiler die Identität erkennt und die Doppelberechnung wegoptimiert.

4.8 Bedingte Ausdrücke

Ein auf den ersten Blick ziemlich kurioses, in der Praxis gelegentlich aber durchaus nützliches Konstrukt ist der *bedingte Ausdruck* (`? :`).

`bedingung ? ausdruck1 : ausdruck2`

Wenn die Bedingung `bedingung` den Wert *wahr* besitzt, ist der Wert des Gesamtausdrucks der Wert von `ausdruck1`, sonst der Wert von `ausdruck2`. Ein zunächst noch mehr formales Beispiel: Statt

```
if (bedingung) {
    x = ausdruck1;
}
else {
    x = ausdruck2;
}
```

kann man kurz

```
x = bedingung ? ausdruck1 : ausdruck2;
```

schreiben. Ein konkretes Beispiel: Durch

```
max = z1 > z2 ? z1 : z2;
```

erhalten wir in der Variablen `max` das Maximum der beiden Zahlen `z1` und `z2`.

Die drei Ausdrücke, die gemeinsam einen bedingten Ausdruck bilden, brauchen auf keinen Fall geklammert zu werden, da die Operatoren `?` und `:` zwar eine höhere Priorität als der Zuweisungsoperator, gleichzeitig aber eine niedrigere Priorität als alle anderen Operatoren haben.

4.9 Der Kommaoperator

Der *Kommaoperator* `(,)` fasst mehrere Ausdrücke zu einem einzigen Ausdruck zusammen.

Beispiel: Die Werte der beiden Variablen `x` und `y` sollen vertauscht werden. Statt

```
h = x;  
x = y;  
y = h;
```

kann man auch

```
h = x, x = y, y = h;
```

schreiben. Dass dieses kürzer scheint, liegt ausschließlich daran, dass jeweils eine Anweisung in eine Zeile geschrieben wurde, was der Standard keineswegs verlangt.

Durch Kommaoperatoren getrennte Ausdrücke werden von links nach rechts ausgewertet. Der Wert des Gesamtausdrucks ist der Wert des am weitesten rechts stehenden, zuletzt ausgewerteten Teilausdrucks.

Der Kommaoperator sollte nur dann verwendet werden, wenn Konstruktionen sehr eng zusammengehören, wie im Beispiel: Den Tausch der Werte von zwei Variablen kann man durchaus als eine Operation ansehen. Wir werden andere Stellen kennenlernen, an denen sich der Kommaoperator als durchaus nützlich erweist.

Kommata, die in Variablenvereinbarungen und Funktionsaufrufen stehen, haben übrigens *nichts* mit dem Kommaoperator zu tun. Entsprechend haben sie dort auch keinen Einfluss auf die Reihenfolge der Auswertung.

4.10 Priorität der Operatoren

Da wir nun schon eine ganze Reihe von Operatoren behandelt haben, ist es an der Zeit, dass wir uns die *Prioritäten* der Operatoren einmal im Zusammenhang ansehen. Die Aufstellung ist in Abbildung 4.2 enthalten.

Als erstes ist die Prioritätsstufe der Operatoren angegeben: Je höher die Stufe, desto höher die Priorität. Wenn auf einer Stufe mehrere Operatoren stehen, so haben sie dieselbe Priorität.

Für jede Prioritätsstufe ist ganz rechts angegeben, ob die Abarbeitung ihrer Operatoren von links nach rechts (z.B. bei Addition und Subtraktion) oder von rechts nach links (z.B. bei Wertzuweisungen) erfolgt.

Auch wenn noch nicht alle Operatoren bekannt sind, lohnt es sich, die Tabelle einmal in Muße zu studieren und sich die Prioritätsstufen im wesentlichen einzuprägen. Allzu extensive Nutzung der Kenntnis der Prioritätsstufen lohnt allerdings auch nicht, da dann

Stufe	Operator	Beschreibung	Auswertungsreihenfolge
15	()	Auswertung von Parametern	→
	[]	Auswahl einer Feldkomponente	
	-> .	Auswahl einer Strukturkomponente	
14	! ~	Negation (logisch, bitweise)	←
	++ --	Inkrementierung, Dekrementierung (Präfix oder Postfix)	
	+ -	Vorzeichen (unär)	
	(Typ)	Typumwandlung	
	& *	Adressbildung, Dereferenzierung (unär)	
	sizeof	Bestimmung des Speicherbedarfs	
13	* /	Multiplikation (binär), Division	→
	%	Rest bei ganzzahliger Division	
12	+ -	Summe, Differenz (binär)	→
11	<< >>	bitweise Verschiebung nach links, rechts	→
10	< <=	Vergleich auf kleiner, kleiner oder gleich	→
	> >=	Vergleich auf größer, größer oder gleich	
9	== !=	Vergleich auf gleich, ungleich	→
8	&	Und (bitweise)	→
7	^	exklusives Oder (bitweise)	→
6		inklusives Oder (bitweise)	→
5	&&	Und (logisch)	→
4		inklusives Oder (logisch)	→
3	? :	bedingte Auswertung (nur paarweise!)	←
2	=	Wertzuweisung	←
	+=	zusammengesetzte Wertzuweisung (auch: -=, *=, /=, <=<, >>=, &=, ^=, =)	
1	,	sequentielle Auswertung	→

Tabelle 4.2: Prioritäten der Operatoren

leicht völlig unleserliche Ausdrücke resultieren. Die folgende Auswahl verdeutlicht diesen Umstand:

```

i - j          /* 1: binaer          */
i -- j         /* 2: unzulaessig         */
i - - j        /* 3: binaer/unaer          */
i --- j        /* 4: Dekrement/binaer (wie 6.) */
i - -- j       /* 5: binaer/Dekrement      */
i -- - j       /* 6: Dekrement/binaer     */
i ---- j       /* 7: unzulaessig          */
i - --- j      /* 8: unzulaessig          */
i -- -- j      /* 9: unzulaessig          */
i --- - j      /* 10: Dekrement/binaer/unaer */
i ----- j     /* 11: unzulaessig         */
i - ----- j  /* 12: unzulaessig         */
i -- --- j     /* 13: unzulaessig         */
i --- -- j     /* 14: Dekrement/binaer/Dekrement */
i ----- - j  /* 15: unzulaessig         */
i ----- j    /* 16: unzulaessig         */

```

Falls ein Ausdruck dieser Art benötigt wird, kann es nicht schaden, ihn auch dann zu klammern, wenn dies nicht zwingend notwendig ist.

4.11 Nebeneffekte

Unter einem *Nebeneffekt* versteht man bei der Programmierung, dass als Nebenprodukt der Auswertung eines Ausdrucks der Wert einer Variablen verändert wird.

In der Regel wird dringend geraten, Nebeneffekte zu vermeiden. Das ist in C jedoch nicht möglich, denn C ist geradezu die „Sprache der Nebeneffekte“: Da die Wertzuweisung keine Anweisung, sondern ein Operator ist, lassen sich die Werte von Variablen *nur* durch Nebeneffekte verändern!

Neben den Zuweisungsoperatoren bewirken auch die Inkrement- und Dekrementoperatoren sowie alle Funktionen mit Ausgabeparametern Nebeneffekte. Beispiel:

```
while (scanf (...) == 1)
    ...
```

Der Standard schreibt nur sehr eingeschränkt vor, zu welchem Zeitpunkt Nebeneffekte wirksam werden müssen:

- Nebeneffekte können natürlich nicht eintreten, bevor die Auswertung des Ausdrucks beginnt, durch die sie erzeugt werden.
- Wenn die Auswertung eines Ausdrucks abgeschlossen ist, müssen auch seine Nebeneffekte eingetreten sein.

Die Folge: Wenn eine Variable innerhalb eines Ausdrucks mehrfach angesprochen wird und außerdem Nebeneffekte für sie eintreten, ist meist *nicht* definiert, welchen Wert die Variable im Einzelfall besitzt. So können Ausdrücke wie

```
v[i] = i++;
v[i] = w[i++];
s += v1[i] * v2[i++];
b = a++ + a;
```

implementationsspezifisch unterschiedliche Resultate zeitigen.

Ähnliche Probleme können sich aus der Tatsache ergeben, dass die Reihenfolge der Auswertung von Ausdrücken nur teilweise festgelegt ist:

- Für die drei Ausdrücke, die gemeinsam einen bedingten Ausdruck bilden, liegt die Reihenfolge eindeutig fest: Erst wird die Bedingung ausgewertet, dann der Ausdruck, der das Resultat ergibt. Der jeweils andere Ausdruck wird nicht ausgewertet und es treten keine Nebeneffekte für ihn ein.
- Logische Ausdrücke werden von links nach rechts ausgewertet; ihre Berechnung wird beendet, sobald das Resultat feststeht. Bei `||` ist das der Fall, sobald der erste Teilausdruck wahr ist, bei `&&` sobald der erste Teilausdruck falsch ist. Nebeneffekte treten jeweils auf, bevor mit dem nächsten Teilausdruck fortgefahren wird.
- Ausdrücke, die mit dem Kommaoperator verknüpft sind, werden von links nach rechts ausgewertet. Dabei treten alle Nebeneffekte auf, bevor der rechte Ausdruck ausgewertet wird.
- Sonst ist zwar festgelegt, in welcher Reihenfolge aufeinanderfolgende Operatoren gleicher Hierarchiestufe ausgeführt werden, nicht aber die Reihenfolge der Berechnung der Operanden.

Betrachten wir für den letzten Punkt ein Beispiel: Der Ausdruck

```
(a) + (b) + (c)
```

muss wie

```
((a) + (b)) + (c)
```

ausgewertet werden. Sind *a*, *b* und *c* ihrerseits Ausdrücke, so ist keineswegs festgelegt, welcher von ihnen zuerst berechnet wird. Konkret: Die Anweisungsfolge

```
i = 5;
j = i * 3 + ++i - 7;
```

kann in der Variablen *j* je nach Implementation den Wert 14 oder 17 liefern.

Ebenfalls ist nicht festgelegt, in welcher Reihenfolge die Argumente eines Funktionsaufrufs berechnet werden. So kann die Anweisungsfolge

```
n = 3;
printf("%d, %d\n", ++n, 2*n);
```

je nach Implementation die Ausgabe 4, 6 oder 4, 8 ergeben.

Aber auch die scheinbar problemlosen Operatoren bieten „Fallstricke“, weil bei ihnen Teilausdrücke unter Umständen nicht ausgewertet werden und in ihnen steckende Nebeneffekte entsprechend mal eintreten und mal nicht:

```
if (x == y && a == ++b)          /* 1 */
    ...
if (x == y || a == ++b)          /* 2 */
    ...
z = x == y ? a : ++b;            /* 3 */
```

Im ersten Beispiel wird *b* nur dann inkrementiert, wenn *x == y* gilt; im zweiten und dritten Beispiel wird *b* nur dann inkrementiert, wenn *x != y* gilt.

Fazit: Bei Operatoren und Funktionen, die Nebeneffekte erzeugen, sollte man *äußerste* Vorsicht walten lassen – zumal man Fehler im Programm, die aus Nebeneffekten resultieren, oft nur mit großer Mühe findet. Insbesondere sollte man darauf achten, dass Variablen, die inkrementiert oder dekrementiert werden, innerhalb eines Ausdrucks nur an der Stelle vorkommen, an der sie inkrementiert bzw. dekrementiert werden.

4.12 Typumwandlung

Einen wesentlicher Punkt ist bisher noch nicht eindeutig geklärt worden: Welche Typen dürfen bzw. müssen die Operanden eines binären arithmetischen Operators besitzen?

Die Antwort ist ganz einfach: Jede beliebige Kombination ist erlaubt! Oder, um den Fachbegriff zu verwenden: *mixed mode* ist uneingeschränkt erlaubt. Das Beispiel, das wir bereits betrachtet hatten, war

```
10.0 / 3 = 3.333333
```

Die Frage ist: Welchen Wert erhalten solche Ausdrücke? Der Rechner kann grundsätzlich nur Operanden mit identischen Typen verknüpfen, so dass also vor die eigentliche Verknüpfung ggf. eine Typumwandlung vorgeschaltet werden muss. Dieses übernimmt der C-Compiler in der Regel für den Programmierer; man bezeichnet solche Typumwandlungen auch als *implizite Typumwandlungen*. Dass das Resultat einer Verknüpfung den Typ der beteiligten Operanden (ggf. nach der Umwandlung) hat, ist naheliegend, von der

ganzzahligen Division vielleicht einmal abgesehen. Man kann deshalb auch vom „Typ eines Ausdrucks“ sprechen.

Schauen wir uns zunächst an, was bei der Verknüpfung von zwei Operanden durch einen binären arithmetischen Operator geschieht:

Der erste Grundsatz: Mit Werten der Typen `char` und `short`, mit oder ohne Vorzeichen, wird nicht gerechnet, d.h. Werte mit diese Typen werden vor dem Rechnen auf jeden Fall automatisch in `int` bzw. `unsigned int` umgewandelt. Ein Ausdruck wie `'Z' - 'A'` hat also den Typ `int`.

Der zweite Grundsatz: Bei Operanden mit unterschiedlichen Typen wird der Operand mit dem „niederwertigen“ Typ in den „höherwertigen“ Typ umgewandelt. Was unter „niederwertig“ und „höherwertig“ zu verstehen ist, legt die in Tabelle 4.3 dargestellte Hierarchie der Typen fest.

long und int ungleich	long und int gleich
7. long double	5. long double
6. double	4. double
5. float	3. float
4. unsigned long	2. unsigned int/long
3. long	1. int/long
2. unsigned int	
1. int	

Tabelle 4.3: Hierarchie der impliziten Typumwandlung

Dass die Typen `char` und `short` hier nicht vorkommen, liegt daran, dass sie ohnehin umgewandelt werden.

Bei komplexeren Ausdrücken entwickelt sich der Typ im Laufe der Auswertung. Insbesondere wird nicht zuerst der Typ des Resultats bestimmt und erst danach die Berechnung ausgeführt. Dahinter steht, dass jeder Teilausdruck im einfachsten möglichen Typ ausgewertet werden soll. Beispiel:

```
short s = 65;
int i = 7;
long j = 1000;
float x = 3.0;
double y = 46.5;

printf("%f\n", s * i + j % i + y / x);
```

Zunächst werden die Multiplikationen und Divisionen ausgeführt, wobei die angegebene Reihenfolge nicht notwendig eingehalten werden muss:

- Im Ausdruck `s * i` wird `s` zunächst in `int` umgewandelt. Die Multiplikation liefert den `int`-Wert 455.
- Im Ausdruck `j % i` wird `i` zunächst in `long` umgewandelt. Der Divisionsrest ist der `long`-Wert 6.
- Im Ausdruck `y / x` wird `x` zunächst in `double` umgewandelt. Der Quotient ist der `double`-Wert 15.5.

Die Additionen *müssen* nun von links nach rechts ausgeführt werden:

- Der `int`-Wert 455 wird in `long` umgewandelt und zum `long`-Wert 6 addiert. Die Summe ist der `long`-Wert 461.

- Der `long`-Wert 461 wird in `double` umgewandelt und zum `double`-Wert 15.5 addiert. Die Summe ist der `double`-Wert 476.5. Das ist gleichzeitig das Gesamtergebn.

Die Art und Weise, in der die impliziten Typumwandlungen erfolgen, birgt wieder „Fallstricke“. Ein häufig gemachter Fehler ist zum Beispiel die unbeabsichtigte ganzzahlige Divisionen. So liefert der Ausdruck `1 / 2 * 3.5` stets den Wert 0! In derart offensichtlicher Form kommt dieser Fehler in der Praxis natürlich selten vor. Häufiger ist er mit zwei `int`-Variablen, etwa `i / j * 3.5` – was an der ganzzahligen Division nichts ändert.

Die Zuweisungsoperatoren nehmen eine Sonderstellung ein! Es ist einer Variablen ein Wert zuzuweisen – der Typ einer Variablen ist aber nichts, was während der Ausführung eines Programms verändert werden kann. Typumwandlung im Zuge einer Wertzuweisung ist also zwar ggf. erforderlich, kann aber stets nur in einer Umwandlung des Typs des Wertes in den Typ der Variablen bestehen. Dabei kann es erforderlich sein, dass aus einem „höherwertigen“ in einen „niederwertigen“ Typ umgewandelt wird:

- Bei der Umwandlung zwischen ganzzahligen Typen werden nur die wertniedrigsten Bits übernommen.
- Bei der Umwandlung eines Gleitkommawertes in einen ganzzahligen Typ gehen eventuelle Nachkommastellen verloren. Eine Rundung erfolgt bei dieser Umwandlung *nicht*! So erhält die `int`-Variable `i` durch die Wertzuweisung `i = 3.9`; den Wert 3!
- Wenn bei Gleitkommatypen aus höherer in niedrigere Genauigkeit umgewandelt wird (z.B. von `double` in `float`), bleibt es den Implementatoren überlassen, ob dabei gerundet oder abgeschnitten wird. (Das ist bei binären Darstellungen letztlich gleichwertig!)
- Bei der Umwandlung aus einem „großen“ ganzzahligen Typ in einen „kleinen“ Gleitkommatyp kann Genauigkeit verlorengehen.

Über die impliziten Typumwandlungen, die ggf. für die Argumente von Funktionen durchgeführt werden, wurde bei der Einführung von `printf` schon einiges gesagt. Im Rahmen der Behandlung der Funktionen wird darauf noch weiter eingegangen.

4.13 Castoperatoren

Gelegentlich ist es wünschenswert, den Compiler dazu zwingen zu können, einen Wert in einen bestimmten Typ umzuwandeln, etwa zur Vermeidung von unerwünschten ganzzahligen Divisionen.

Hierfür stellt C die *Castoperatoren* zur Verfügung. Solch ein Castoperator ist eine Typbezeichnung, die in runde Klammern eingeschlossen wird. Castoperatoren sind immer unär und haben sehr hohe Priorität, wie alle anderen unären Operatoren.

Wollen wir etwa erreichen, dass der `double`-Quotient von zwei `int`-Variablen `i` und `j` berechnet wird, so können wir alternativ schreiben

```
(double) i / j
i / (double) j
```

Der Castoperator wirkt dabei wegen seiner hohen Priorität jeweils nur auf einen der beiden Operanden. Der andere Operand wird dann gemäß der Regeln der impliziten Typumwandlung ebenfalls nach `double` umgewandelt.

Alle Probleme der Umwandlungen, die durch Wertzuweisungen erzwungen werden, können offensichtlich auch bei der Verwendung von Castoperatoren auftreten. Darüber hinaus erlauben die Castoperatoren bei unsachgemäßer Verwendung auch viel Unsinn. Dies wird besonders später bei der Verwendung von Zeigern deutlich werden.

4.14 Der `sizeof`-Operator

Mit dem `sizeof`-Operator lässt sich der Speicherbedarf für Datentypen ermitteln. Das wird später noch eine Rolle spielen, wenn dynamisch Speicher bereitgestellt werden soll.

```
sizeof ausdruck
sizeof (typ)
```

Im ersten Fall ist das Resultat der Speicherbedarf der benötigt würde, um eine einfache Variable vom Typ des Ausdrucks zu speichern. Im zweiten Fall ist das Resultat der Speicherbedarf einer einfachen Variablen mit dem angegebenen Typ. Die Maßeinheit ist rechnerpezifisch, wird jedoch dadurch festgelegt, dass der Standard `sizeof (char) == 1` vorschreibt. Festgelegt ist außerdem, dass das Resultat den Typ `size_t` besitzt – oder umgekehrt: Der Typ `size_t` ist so zu definieren, dass sein Wertebereich alle möglichen Resultate des Operators `sizeof` umfasst. `size_t` ist demnach kein elementarer Datentyp, sondern wird in den Headerdateien `stddef.h`, `stdlib.h` und `string.h` deklariert.

Bei der Verwendung des `sizeof`-Operators sind einige Dinge besonders zu beachten:

- Handelt es sich bei *ausdruck* um den Namen eines Feldes, bzw. bei *typ* um einen Feldtyp, so wird sowohl die Dimension des Feldes als auch die Größe der einzelnen Elemente berücksichtigt.

```
int field[7];
size_t s;
s = sizeof (field);           /* s == 28 */
s = sizeof (char [2][5]);    /* s == 10 */
```

- Der `sizeof`-Operator ist trotz seiner Form nicht mit einem Funktionsaufruf wie z.B. bei `printf(...)` zu verwechseln. Ganz speziell gilt, dass keine Nebeneffekte für den Ausdruck *ausdruck* auftreten. Der Compiler ermittelt lediglich den *Typ* des Ausdrucks. Zur Laufzeit des Programms werden keine weiteren Operationen ausgeführt. Das kann zu einiger Verwirrung führen:

```
int s = 0;
printf("%d\n", sizeof (5 / s)); /* Ausgabe: 4 */
printf("%d\n", sizeof (s = 5)); /* Ausgabe: 4 */
                               /* s == 0 */
```

Es sind also sogar Ausdrücke erlaubt, die zur Laufzeit einen Fehler verursachen würden (im Beispiel Division mit 0). Da dieser Ausdruck jedoch formal den Typ `int` besitzt, wird der Wert 4 ermittelt (vorausgesetzt der Typ `int` besitzt die Größe 4, was jedoch auf gängigen 32-Bit Architekturen der Fall ist).

